



COMPSCI 130 S1 2022

Introduction to Software Fundamentals

Binary Trees – Tree Traversal



Agenda & Reading

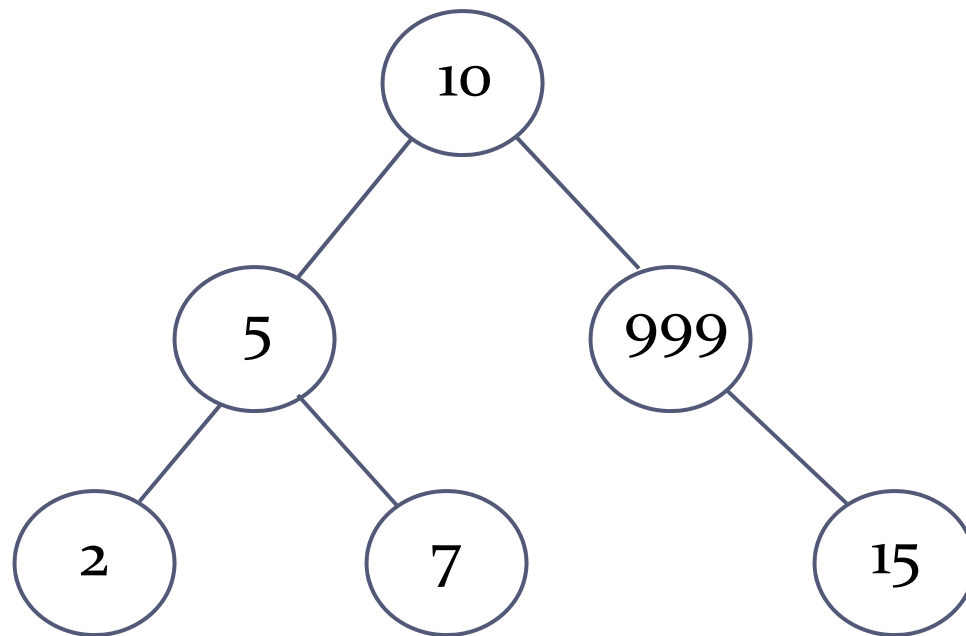
- ▶ Agenda
 - ▶ Tree traversal

- ▶ Online Coursebook:
 - ▶ <https://onlinetextbook.auckland.ac.nz/19-binary-trees/tree-traversals/>



Printing A Tree

- How can we print this tree?

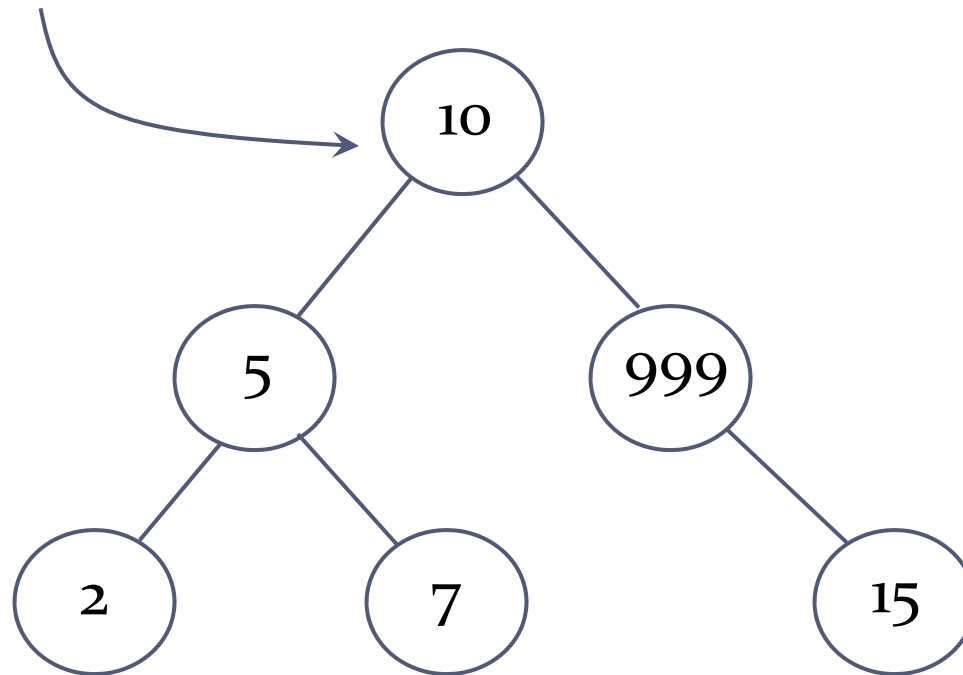




Printing the tree

- How can we print this tree?

Step 1: Print the root node

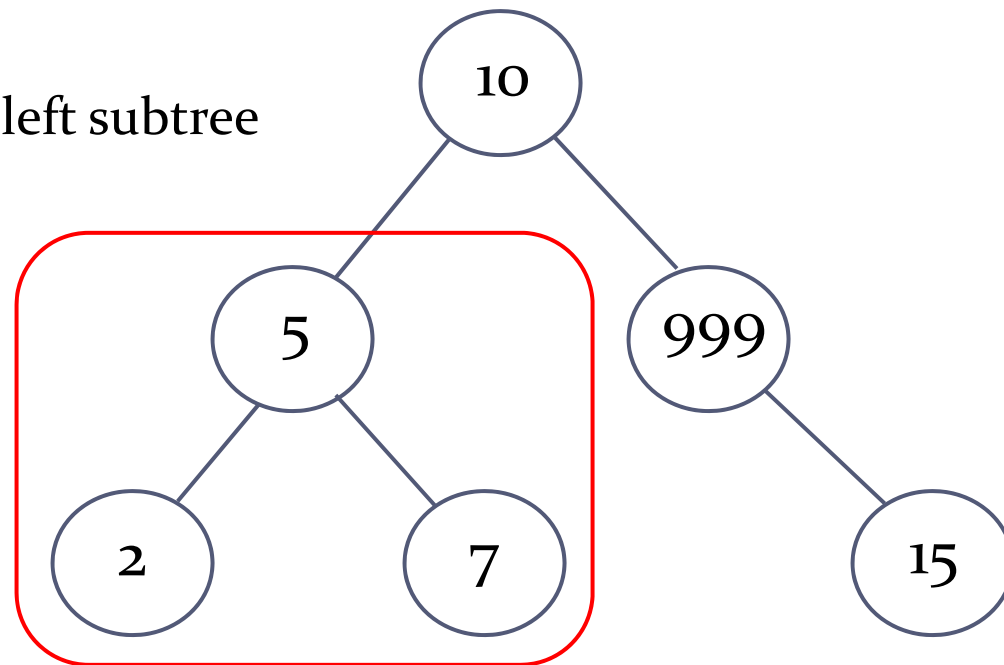




Printing the tree

- How can we print this tree?

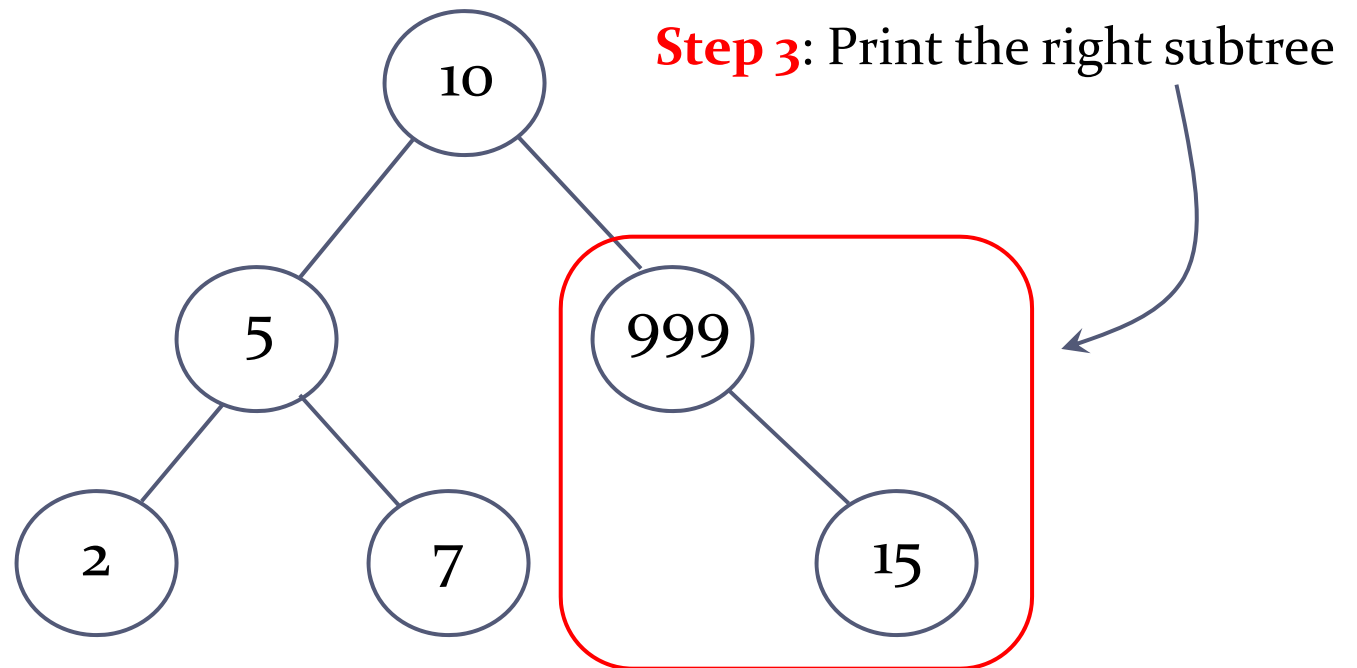
Step 2: Print the left subtree





Printing the tree

- How can we print this tree?

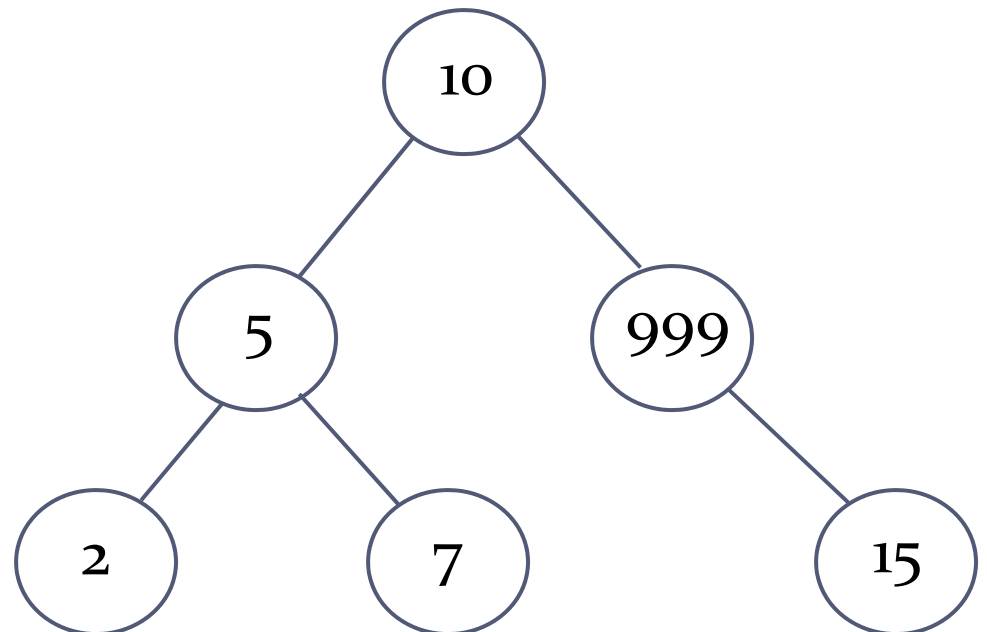




Tree traversals

- ▶ What if we try something like this?

```
def basic_print(tree):  
    if tree != None:  
        print(tree.get_data(), end=" ")  
        basic_print(tree.get_left())  
        basic_print(tree.get_right())
```



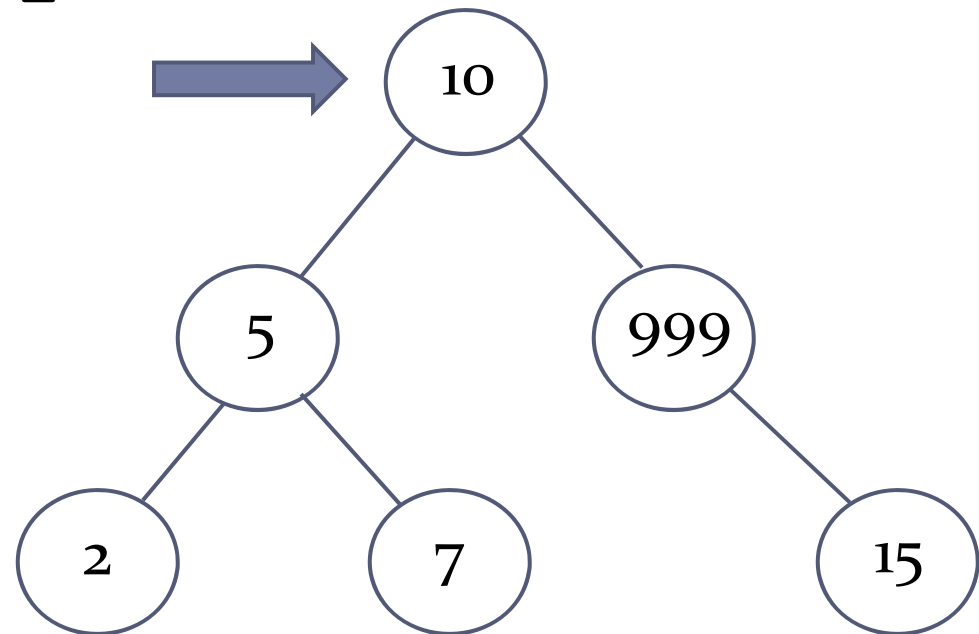


Tree traversals

► What if we try something like this?

```
def basic_print(tree):  
    if tree != None:  
        print(tree.get_data(), end=" ")  
        basic_print(tree.get_left())  
        basic_print(tree.get_right())
```

10



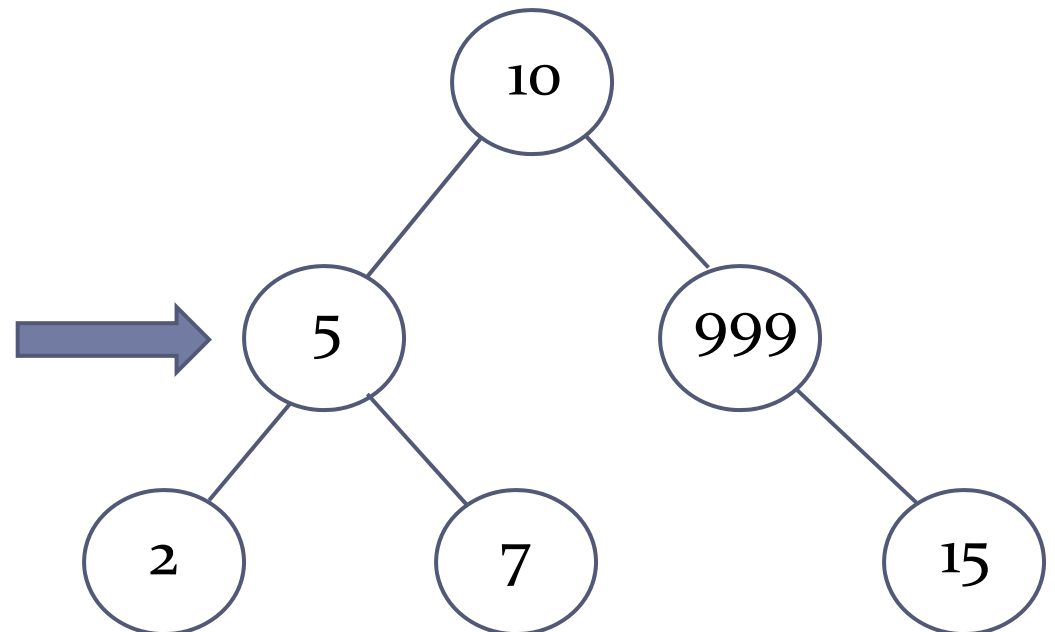


Tree traversals

- ▶ What if we try something like this?

```
def basic_print(tree):  
    if tree != None:  
        print(tree.get_data(), end=" ")  
        basic_print(tree.get_left())  
        basic_print(tree.get_right())
```

10 5



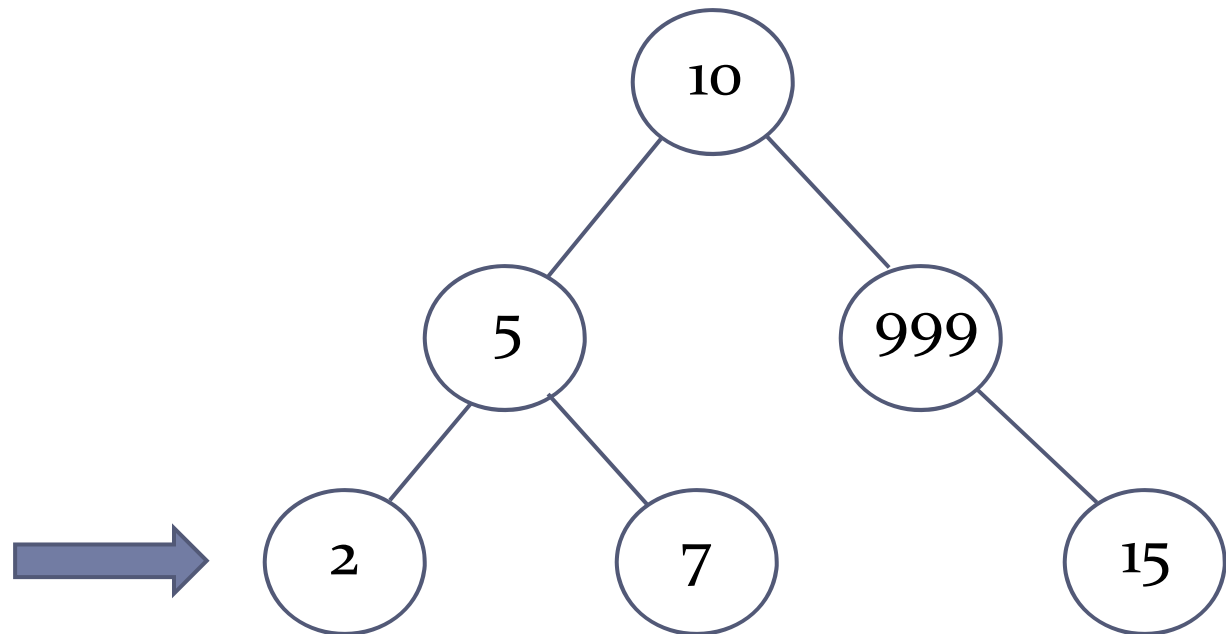


Tree traversals

- ▶ What if we try something like this?

```
def basic_print(tree):  
    if tree != None:  
        print(tree.get_data(), end=" ")  
        basic_print(tree.get_left())  
        basic_print(tree.get_right())
```

10 5 2



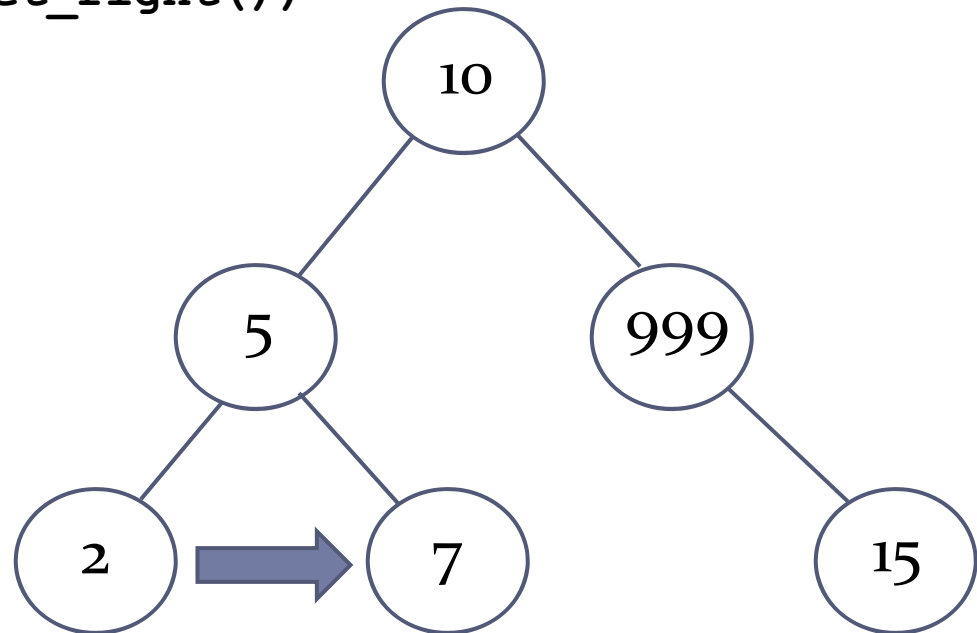


Tree traversals

- ▶ What if we try something like this?

```
def basic_print(tree):  
    if tree != None:  
        print(tree.get_data(), end=" ")  
        basic_print(tree.get_left())  
        basic_print(tree.get_right())
```

10 5 2 7



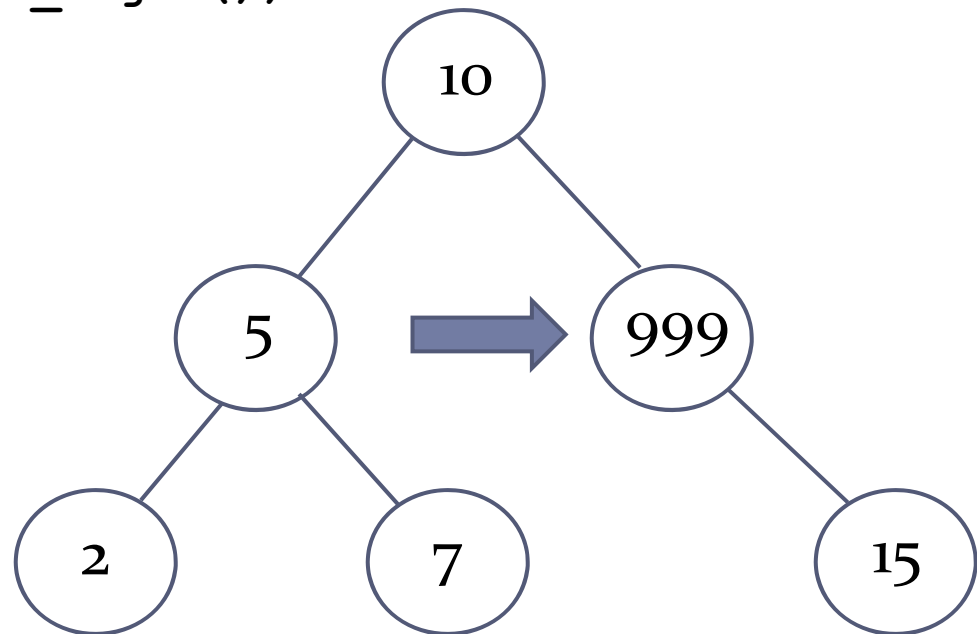


Tree traversals

► What if we try something like this?

```
def basic_print(tree):  
    if tree != None:  
        print(tree.get_data(), end=" ")  
        basic_print(tree.get_left())  
        basic_print(tree.get_right())
```

10 5 2 7 999



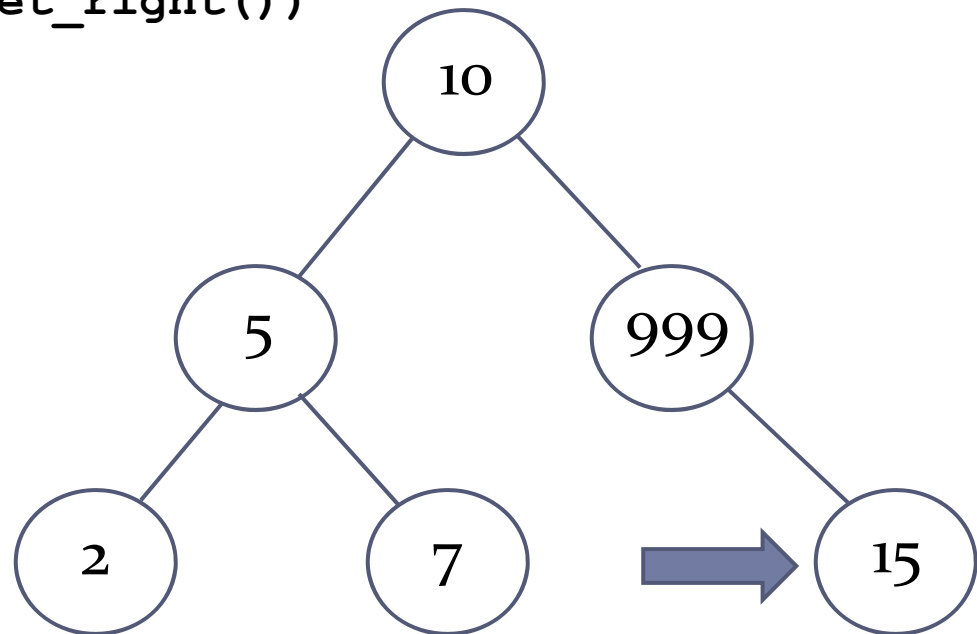


Tree traversals

- What if we try something like this?

```
def basic_print(tree):  
    if tree != None:  
        print(tree.get_data(), end=" ")  
        basic_print(tree.get_left())  
        basic_print(tree.get_right())
```

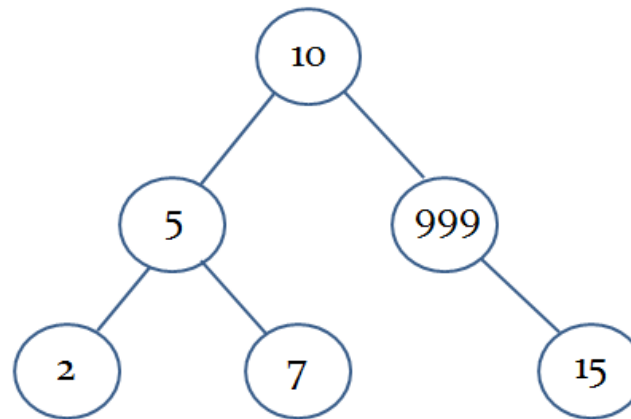
10 5 2 7 999 15





Tree traversals

- ▶ The order in which we print the nodes depends on *where* we place the `print()` statement relative to the recursive calls



```
def basic_print(tree):  
    if tree != None:  
        print(tree.get_data())  
        basic_print(tree.get_left())  
        basic_print(tree.get_right())
```

10 5 2 7 999 15

pre-order

```
def basic_print(tree):  
    if tree != None:  
        basic_print(tree.get_left())  
        print(tree.get_data())  
        basic_print(tree.get_right())
```

2 5 7 10 999 15

in-order

```
def basic_print(tree):  
    if tree != None:  
        basic_print(tree.get_left())  
        basic_print(tree.get_right())  
        print(tree.get_data())
```

2 7 5 15 999 10

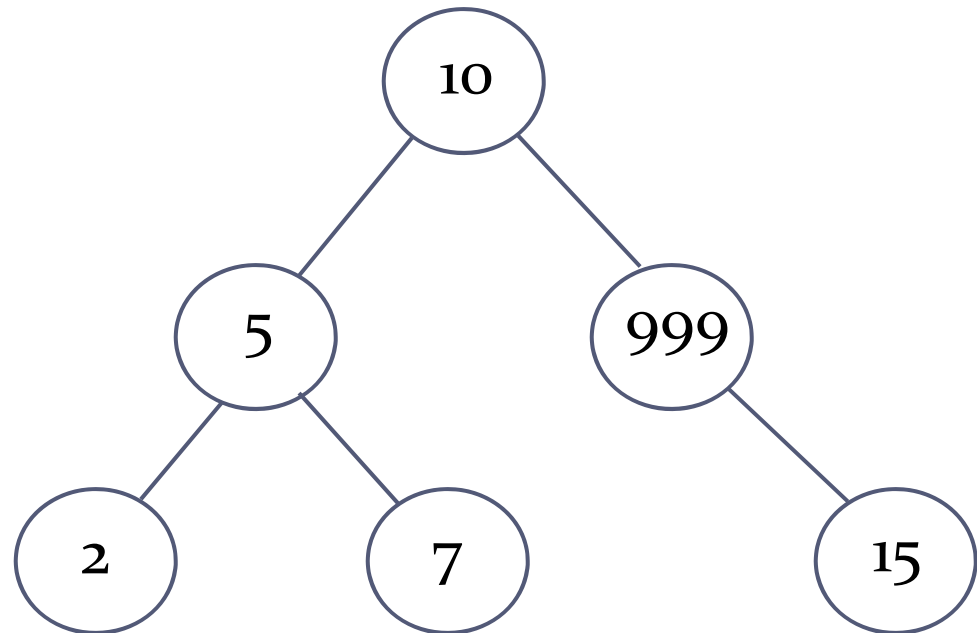
post-order



Tree traversals – In order

► *In order* processing:

```
def basic_print(tree):  
    if tree != None:  
        basic_print(tree.get_left())  
        print(tree.get_data(), end=" ")  
        basic_print(tree.get_right())
```



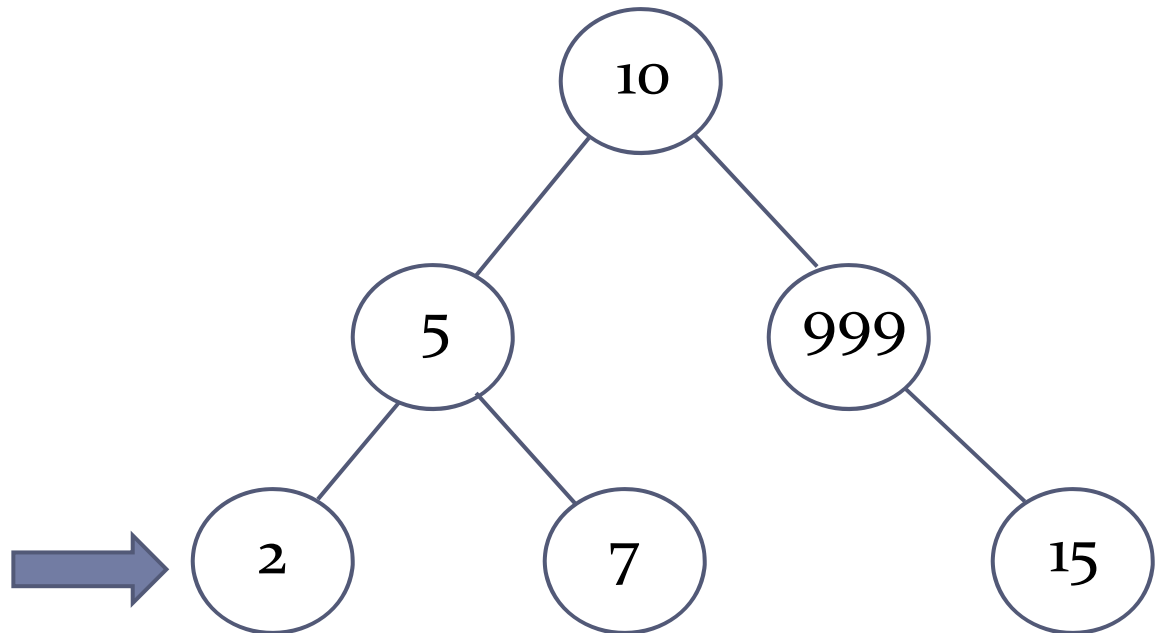


Tree traversals – In order

► In order processing:

```
def basic_print(tree):  
    if tree != None:  
        basic_print(tree.get_left())  
        print(tree.get_data(), end=" ")  
        basic_print(tree.get_right())
```

2



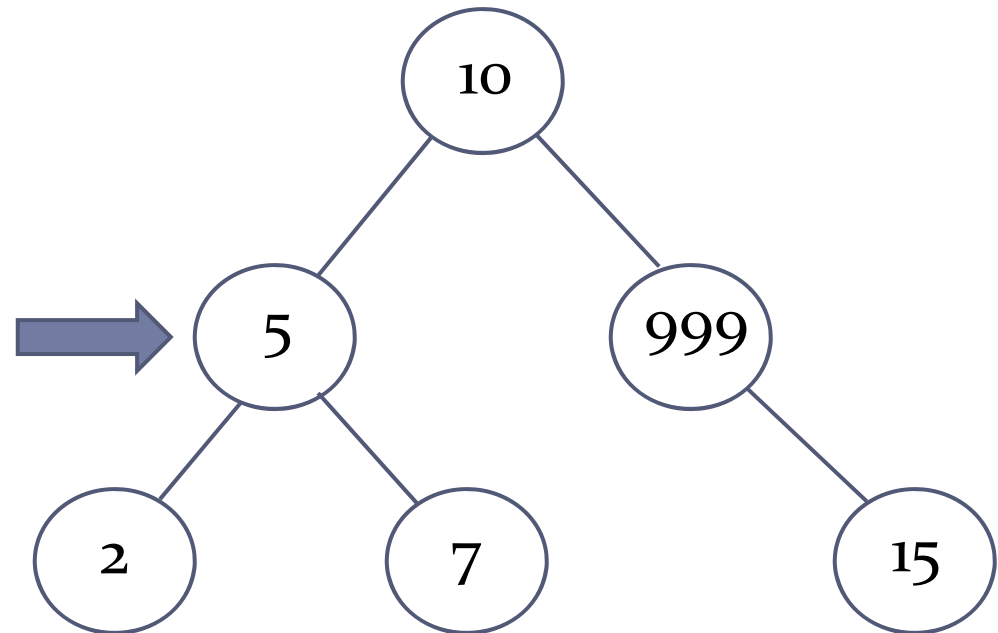


Tree traversals – In order

► *In order* processing:

```
def basic_print(tree):  
    if tree != None:  
        basic_print(tree.get_left())  
        print(tree.get_data(), end=" ")  
        basic_print(tree.get_right())
```

2 5



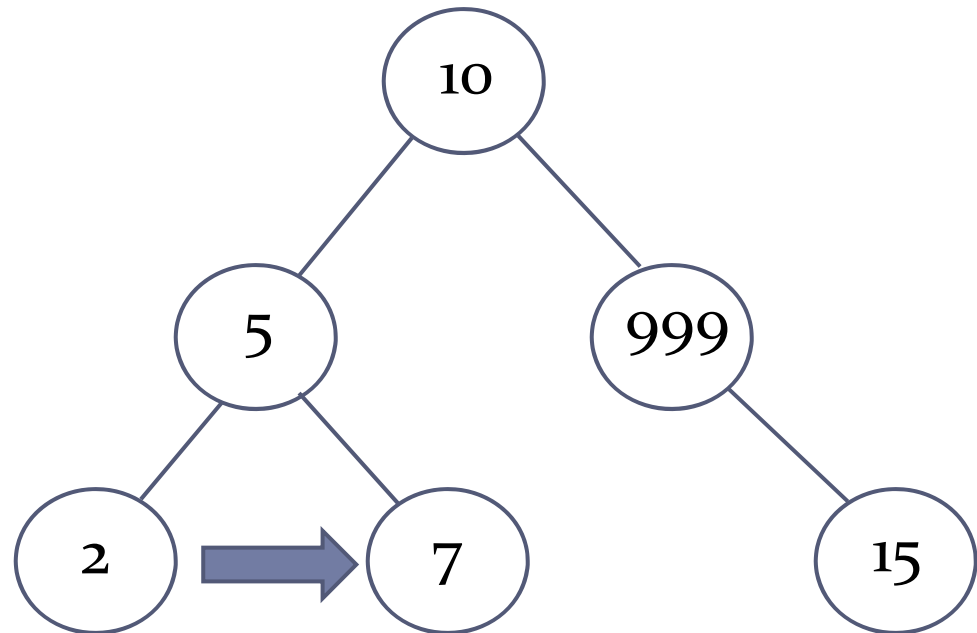


Tree traversals – In order

► *In order* processing:

```
def basic_print(tree):  
    if tree != None:  
        basic_print(tree.get_left())  
        print(tree.get_data(), end=" ")  
        basic_print(tree.get_right())
```

2 5 7



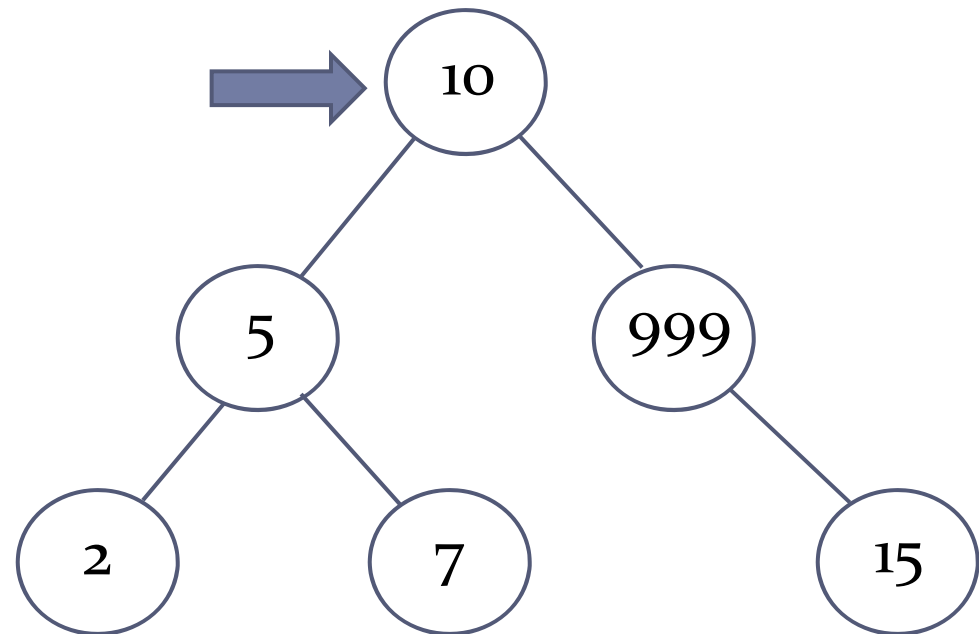


Tree traversals – In order

► *In order* processing:

```
def basic_print(tree):  
    if tree != None:  
        basic_print(tree.get_left())  
        print(tree.get_data(), end=" ")  
        basic_print(tree.get_right())
```

2 5 7 10



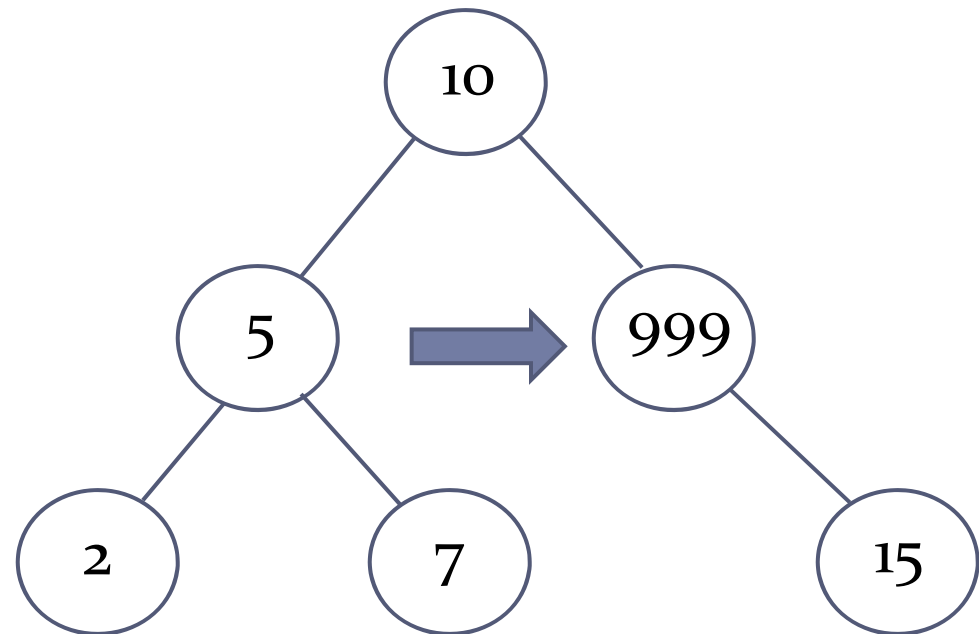


Tree traversals

► *In order* processing:

```
def basic_print(tree):  
    if tree != None:  
        basic_print(tree.get_left())  
        print(tree.get_data(), end=" ")  
        basic_print(tree.get_right())
```

2 5 7 10 999



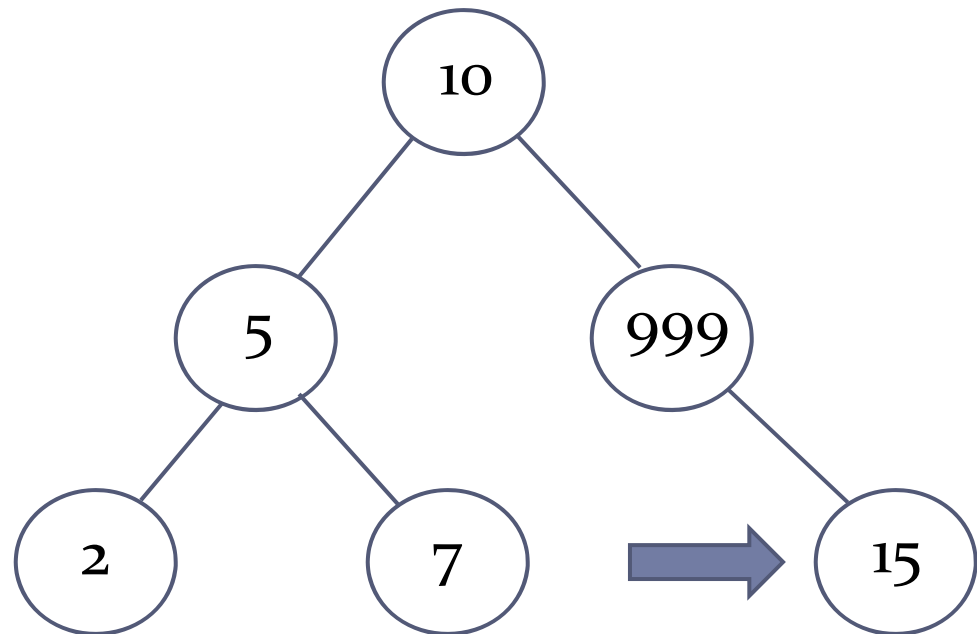


Tree traversals

► In order processing:

```
def basic_print(tree):  
    if tree != None:  
        basic_print(tree.get_left())  
        print(tree.get_data(), end=" ")  
        basic_print(tree.get_right())
```

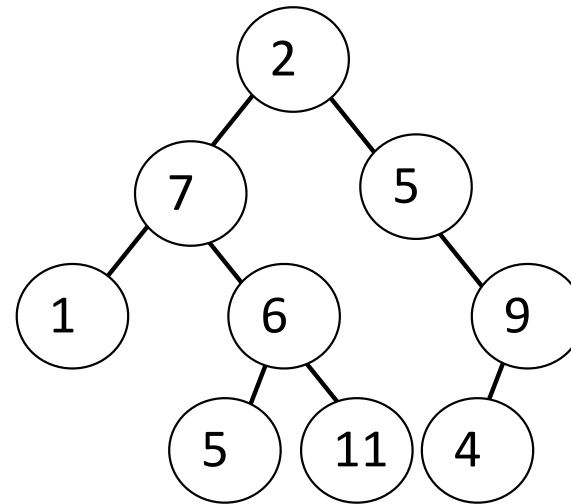
2 5 7 10 999 15





Question

- Write down the pre-order, in-order, and post-order traversals of the tree below



pre-order 2 7 1 6 5 11 5 9 4

in-order 1 7 5 6 11 2 5 4 9

post-order 1 5 11 6 7 4 9 5 2



Exercise 5 (DIY)

- Write down the pre-order, in-order, and post-order traversals of the tree below

